Laboratory 05 – Databases and Network Protocols – Cloud

Sara Viviana Arteaga Rodríguez Julián Felipe Morales Zambrano Diego Alejandro Mesa Alonso

Introducción

En este laboratorio, nos centramos en profundizar nuestra comprensión de cómo se instalan, configuran e integran los componentes fundamentales de la infraestructura de TI para dar soporte a las operaciones comerciales reales. En concreto, el ejercicio se centró en la instalación y configuración del Sistema de Gestión de Bases de Datos (DBMS) PostgreSQL en una máquina virtual Linux Slackware. Esta actividad simuló un entorno empresarial común, donde coexisten servidores físicos y virtuales dentro de topologías de red complejas, incluyendo conexiones cableadas e inalámbricas gestionadas mediante conmutadores, enrutadores y otros dispositivos de red. Nuestro objetivo era replicar el proceso que seguiría una organización para implementar un servidor de base de datos funcional y seguro, capaz de almacenar y organizar información estructurada para su uso futuro por diversas aplicaciones y servicios.

A través de este laboratorio, continuamos adquiriendo experiencia práctica en la gestión no solo de la instalación de software, sino también de la configuración de usuarios, la creación de bases de datos y la inserción de datos. Creamos cuentas de usuario personalizadas, asegurando que cada una tuviera acceso únicamente a sus respectivas bases de datos, reforzando así el principio de control de acceso y aislamiento de datos en entornos multiusuario. Las bases de datos diseñadas en esta actividad se centraron en la catalogación de sitios turísticos en Colombia, lo que nos permitió aplicar conceptos de modelado de bases de datos como el diseño de tablas relacionales, las restricciones de clave externa y la integridad referencial.

Además, este laboratorio enfatizó la importancia de comprender cómo interactúan los protocolos de red con los sistemas de bases de datos. En una infraestructura corporativa, las bases de datos rara vez operan de forma aislada; se comunican constantemente con otros sistemas y servicios a través de la red. Por lo tanto, aprender sobre los protocolos DNS, HTTP y Ethernet utilizando Wireshark nos permitió observar el flujo de comunicación entre clientes y servidores en tiempo real. Esta comprensión cierra la brecha entre el conocimiento teórico de redes y las habilidades prácticas de administración de sistemas. En esencia, el laboratorio nos ayudó a fortalecer nuestra base técnica tanto para la gestión de bases de datos como para la comunicación en red, dos pilares de los entornos de tecnología de la información modernos que garantizan la accesibilidad, la consistencia y la confiabilidad de los datos en todos los sistemas.

I. Review of Network Protocols

I-A. *Review of DNS Messages*

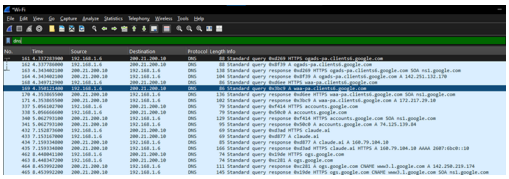


Figura 1: Captura DNS – consulta (paquete 162).

Figura 2: Detalle del paquete DNS de consulta.

El paquete 162 corresponde a una consulta DNS estándar enviada desde el dispositivo con IP 192.168.1.6 hacia el servidor DNS 200.21.200.10. El Transaction ID `0x8f39` identifica de forma única esta consulta para poder emparejarla con su respuesta. Las Flags tienen valor `0x0100`, lo que indica que el mensaje es una petición (*query*) y no una respuesta. El campo Opcode en 0 confirma que es una consulta estándar. El bit de *Truncated* está en 0, lo que significa que el mensaje no fue cortado y llegó completo. El bit *Recursion Desired* está activo, indicando que el cliente le solicita al servidor DNS que realice la búsqueda de forma recursiva si no conoce la respuesta directamente. El campo Questions vale 1, correspondiente al dominio consultado `ogads-pa.clients6.google.com` con tipo A, es decir, se solicita su dirección IPv4. Los campos Answer RRs, Authority RRs y Additional RRs valen 0 porque al ser una petición aún no hay respuesta.

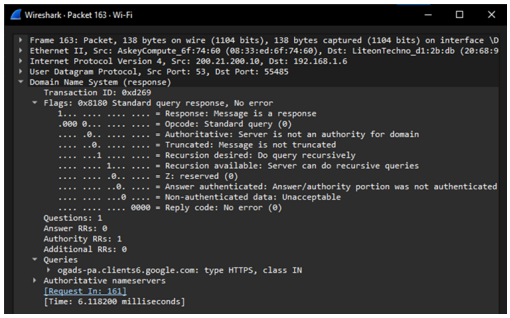


Figura 3: Detalle del paquete DNS de respuesta (paquete 163).

El paquete 163 es la respuesta DNS enviada por el servidor 200.21.200.10 de vuelta al dispositivo 192.168.1.6. El Transaction ID `0x0d269` empareja esta respuesta con su consulta original. Las Flags tienen valor `0x8180`, indicando que es una respuesta sin errores, confirmado por el Reply Code en 0 (No error). El bit *Response* está en 1, confirmando que es una respuesta. El campo *Authoritative* vale 0, lo que significa que el servidor DNS que responde no es el servidor autoritativo del dominio, sino un servidor intermediario. El bit *Truncated* está en 0, el mensaje llegó completo. Los bits *Recursion Desired* y *Recursion Available* están ambos activos, indicando que tanto el cliente pidió recursión como que el servidor tiene esa capacidad. El campo Questions vale 1 y la consulta corresponde a `ogads-pa.clients6.google.com` con tipo HTTPS. Authority RRs vale 1, lo que significa que el servidor devolvió información sobre quién tiene autoridad sobre el dominio consultado.

I-B. Review of HTTP Messages

Figura 4: Captura HTTP – listado de paquetes.

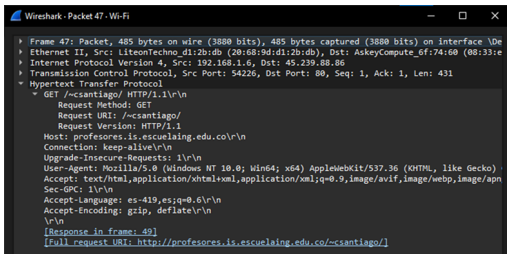


Figura 5: Detalle del paquete HTTP GET (paquete 47).

El paquete 47 corresponde a una solicitud HTTP de tipo GET enviada desde el dispositivo con IP 192.168.1.6 hacia el servidor 45.239.88.86. El método GET indica que el cliente está solicitando obtener un recurso del servidor, en este caso la ruta `/~csantiago/` especificada en el campo Request URI. La versión del protocolo utilizada es HTTP/1.1, que es la versión estándar que permite conexiones persistentes. El campo *Host* indica que la solicitud va dirigida al servidor `profesores.is.escuelaing.edu.co`. El campo *Connection* con valor `keep-alive` le indica al servidor que mantenga la conexión abierta. El campo *Upgrade-Insecure-Requests* en 1 indica que el navegador preferiría una conexión segura HTTPS. El *User-Agent* identifica al cliente como un navegador Mozilla/5.0 corriendo en Windows 10 de 64 bits. El campo *Accept-Language* indica que el cliente prefiere respuestas en español latinoamericano. Finalmente, *Accept-Encoding* con valores gzip y deflate informa al servidor qué métodos de compresión puede manejar el cliente.

I-C. Review of Ethernet Frames

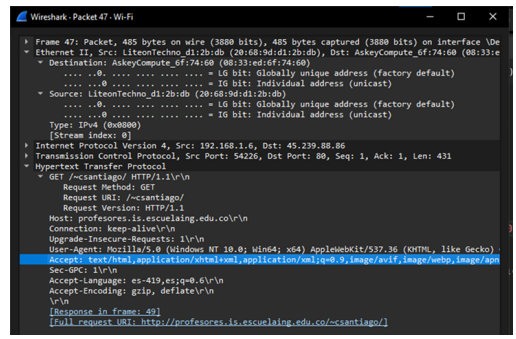


Figura 6: Detalle del encabezado Ethernet II del paquete 47.

El paquete 47 permite examinar el encabezado Ethernet II que encapsula toda la comunicación. El campo *Destination* contiene la dirección MAC `08:33:ed:6f:74:60`, correspondiente al dispositivo AskeyCompute, que es el router o gateway de la red local. El campo *Source* contiene la dirección MAC `20:68:9d:d1:2b:db`, correspondiente al dispositivo LiteonTechno. Ambas direcciones tienen el bit LG en 0, lo que indica que son direcciones MAC globalmente únicas asignadas de fábrica, y el bit IG en 0, confirmando que son direcciones individuales de tipo unicast. El campo *Type* tiene valor `0x0800`, que identifica que el protocolo encapsulado dentro del frame Ethernet es IPv4.

II. Base Software Installation

II-A. PostgreSQL – Linux Slackware

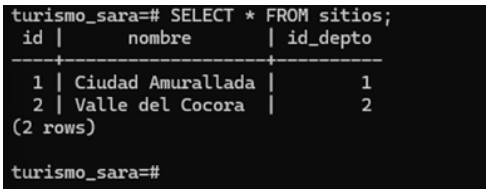
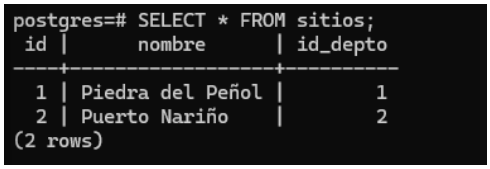
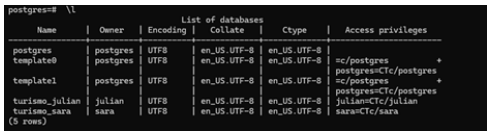


Figura 7: Instalación de PostgreSQL en Slackware (paso 1).



```
postgres=# SELECT * FROM sitios;
 id |       nombre       | id_depto
----+--------------------+----------
  1 | Piedra del Peñol   |        1
  2 | Puerto Nariño      |        2
(2 rows)
```

Figura 8: Instalación de PostgreSQL en Slackware (paso 2).



```
postgres=# \l
                                   List of databases
     Name      | Owner  | Encoding |  Collate   |   Ctype    |   Access privileges
---------------+--------+----------+------------+------------+-----------------------
 postgres      | postgres | UTF8     | en_US.UTF-8 | en_US.UTF-8 |
 template0     | postgres | UTF8     | en_US.UTF-8 | en_US.UTF-8 | =c/postgres          +
               |        |          |            |            |                       | postgres=CTc/postgres
 template1     | postgres | UTF8     | en_US.UTF-8 | en_US.UTF-8 | =c/postgres          +
               |        |          |            |            |                       | postgres=CTc/postgres
 turismo_julian | julian | UTF8     | en_US.UTF-8 | en_US.UTF-8 | julian=CTc/julian
 turismo_sara  | sara   | UTF8     | en_US.UTF-8 | en_US.UTF-8 | sara=CTc/sara
(5 rows)
```

Figura 9: Instalación de PostgreSQL en Slackware (paso 3).

II-B. SQL Server – Windows Server

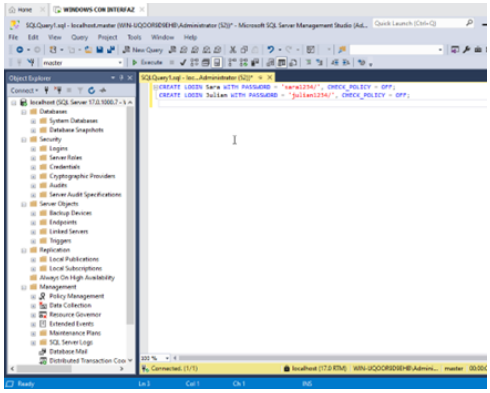
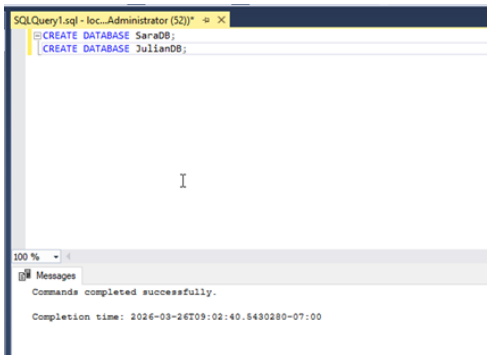


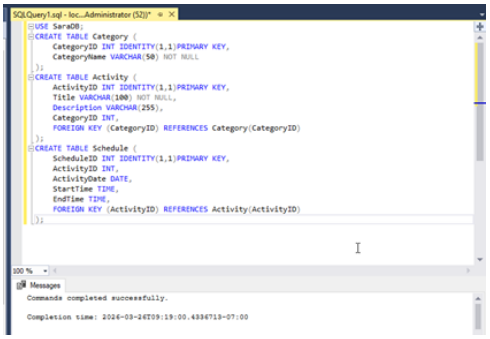
Figura 10: Instalación de SQL Server en Windows Server (paso 1).



```
CREATE DATABASE SaraDB;
CREATE DATABASE JulianDB;
```

```
Commands completed successfully.

Completion time: 2026-03-26T09:02:40.5430280-07:00
```

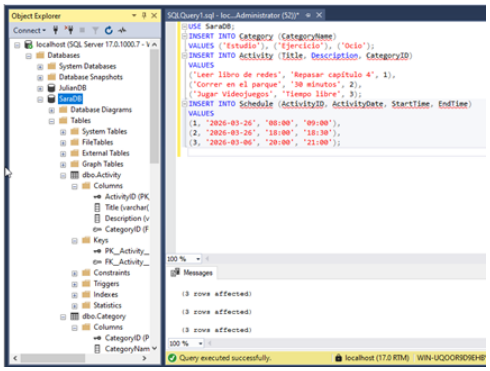
Figura 11: Instalación de SQL Server en Windows Server (paso 2).



```
USE SaraDB;
CREATE TABLE Category (
    CategoryID INT IDENTITY(1,1)PRIMARY KEY,
    CategoryName VARCHAR(50) NOT NULL
);
CREATE TABLE Activity (
    ActivityID INT IDENTITY(1,1)PRIMARY KEY,
    Title VARCHAR(100) NOT NULL,
    Description VARCHAR(255),
    CategoryID INT,
    FOREIGN KEY (CategoryID) REFERENCES Category(CategoryID)
);
CREATE TABLE Schedule (
    ScheduleID INT IDENTITY(1,1)PRIMARY KEY,
    ActivityID INT,
    ActivityDate DATE,
    StartTime TIME,
    EndTime TIME,
    FOREIGN KEY (ActivityID) REFERENCES Activity(ActivityID)
);
```

```
Commands completed successfully.

Completion time: 2026-03-26T09:19:00.4336713-07:00
```

Figura 12: Instalación de SQL Server en Windows Server (paso 3).



```
USE SaraDB;
INSERT INTO Category (CategoryName)
VALUES ('Estudio'), ('Ejercicio'), ('Ocio');
INSERT INTO Activity (Title, Description, CategoryID)
VALUES
('Leer libro de redes', 'Repasar capítulo 4', 1),
('Correr en el parque', '30 minutos', 2),
('Jugar videojuegos', 'Tiempo libre', 3);
INSERT INTO Schedule (ActivityID, ActivityDate, StartTime, EndTime)
VALUES
(1, '2026-03-26', '08:00', '09:00'),
(2, '2026-03-26', '18:00', '18:30'),
(3, '2026-03-06', '20:00', '21:00');
```

```
(3 rows affected)
(3 rows affected)
(3 rows affected)
```

Figura 13: Instalación de SQL Server en Windows Server (paso 4).

II-C. SQL Database – Microsoft Azure

Primero, ingresamos a la página con nuestro correo y después le damos en crear recurso y le damos en crear bases de datos SQL.

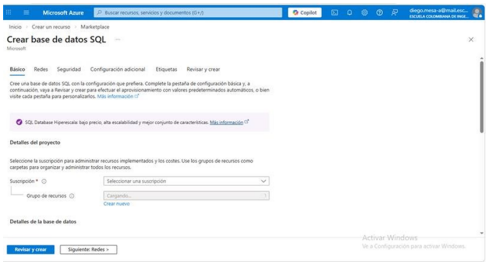


Figura 14: Creación de recurso en Microsoft Azure.

Configuramos la suscripción, el grupo de recursos, el nombre de la base de datos y añadimos un nombre al servidor.

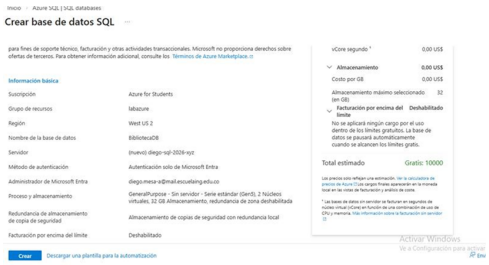


Figura 15: Configuración de la base de datos en Azure.

Creamos la base de datos en nuestro computador.

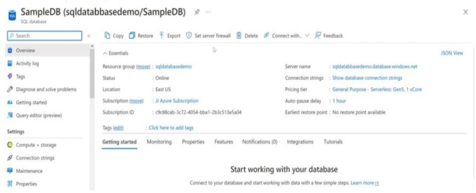


Figura 16: Base de datos creada localmente.

Creamos las 3 tablas e insertamos los datos.

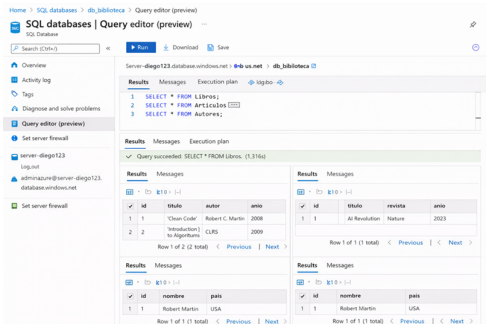


Figura 17: Tablas creadas e inserción de datos en Azure SQL.

II-C1. ¿Qué es cloud computing y ventajas vs on-premise?: Cloud computing es el uso de servidores y servicios por internet en lugar de infraestructura local.

Ventajas de Azure vs on-premise: Escalabilidad bajo demanda, menor costo inicial, alta disponibilidad y respaldo, acceso desde cualquier lugar, mantenimiento gestionado por Microsoft.

II-C2. Servicios de Azure (IaaS, PaaS, SaaS):

- **IaaS:** Infraestructura (VMs, redes). El usuario administra el SO y las apps.
- **PaaS:** Plataforma (ej. Azure SQL). Solo se gestiona la aplicación.
- **SaaS:** Software listo (ej. Microsoft 365). No se gestiona nada técnico.

II-C3. Regiones y Availability Zones:

- **Regiones:** Ubicaciones geográficas de datacenters.
- **Zonas:** Datacenters separados dentro de una región.

II-C4. Escalado vertical vs horizontal:

- **Vertical:** Aumentar recursos de una sola máquina (CPU/RAM).
- **Horizontal:** Agregar más máquinas.

II-C5. TLS en Azure SQL vs local: Azure SQL usa TLS por defecto – conexión cifrada y segura. En local, puede no estar habilitado – menos seguridad si no se configura.

II-C6. TCP: VM local vs Azure SQL:

- **VM local:** Conexión directa, control total del servidor.
- **Azure SQL:** Conexión a servicio gestionado – usa gateways, balanceo y optimización automática de conexiones.

II-D. PostgreSQL – NetBSD

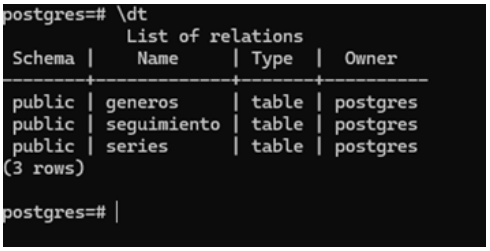


Figura 18: Instalación de PostgreSQL en NetBSD (paso 1).

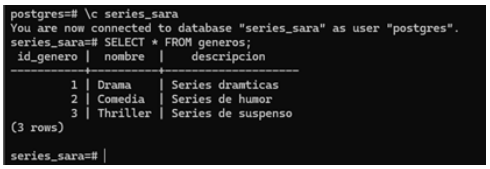


Figura 19: Instalación de PostgreSQL en NetBSD (paso 2).

Figura 20: Instalación de PostgreSQL en NetBSD (paso 3).

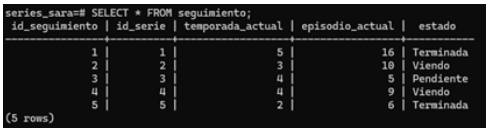


Figura 21: Instalación de PostgreSQL en NetBSD (paso 4).

III. Other Database Engine Configurations

III-A. Inicio automático al arranque del SO

En los servidores donde se instalaron los motores de base de datos, se configuró el sistema operativo para que los motores se inicien automáticamente al arrancar.

Slackware:

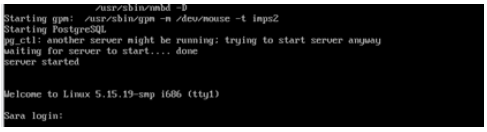


Figura 22: Configuración de inicio automático en Slackware.

Windows:

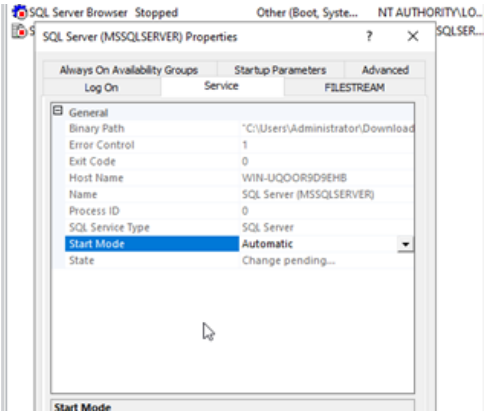


Figura 23: Configuración de inicio automático en Windows (paso 1).

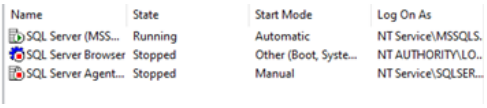


Figura 24: Configuración de inicio automático en Windows (paso 2).

III-B. Conexión remota con DBeaver

Usando DBeaver como cliente de base de datos, se conectó a las bases de datos desde una máquina remota para visualizar el contenido de las tablas.

Slackware:

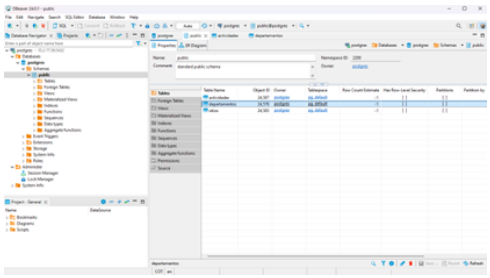


Figura 25: Conexión remota con DBeaver a Slackware.

Windows:

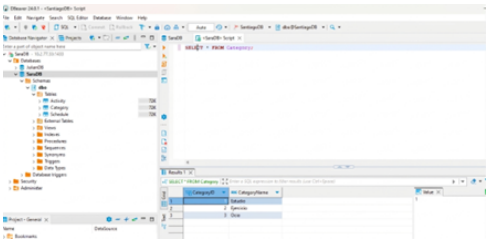


Figura 26: Conexión remota con DBeaver a Windows Server.

Anexos

Instalación de la base de datos en Slackware

```
root@Sara:~/postgresql# wget https://slackbuilds.org/slackbuilds/15.0/system/postgresql.tar.gz
```

```
root@Sara:~/postgresql# tar -xvf postgresql.tar.gz
```

```
root@Sara:~/postgresql# cd postgresql/
```

```
root@Sara:~/postgresql# ./postgresql.SlackBuild
```

```
root@Sara:~/postgresql# cat postgresql.info
PRGNAM="postgresql"
VERSION="14.22"
HOMEPAGE="https://www.postgresql.org"
DOWNLOAD="https://ftp.postgresql.org/pub/source/v14.22/postgresql-14.22.tar.gz"
MD5SUM="2e56b56f7f8670c89e4d00104d926007"
DOWNLOAD_x86_64=""
MD5SUM_x86_64=""
REQUIRES=""
MAINTAINER="Dionysis Ntreou"
EMAIL="yakizarns1@gmail.com"
root@Sara:~/postgresql# wget https://ftp.postgresql.org/pub/source/v14.22/postgresql-14.22.tar.gz
```

```
root@Sara:~/postgresql# groupadd -g 209 postgres
```

```
root@Sara:~/postgresql# useradd -u 209 -g 209 -d /var/lib/pgsql -s /bin/bash postgres
```

```
root@Sara:~/postgresql# ls
README      doinst.sh              postgresql.SlackBuild*  postgresql.logrotate  setup.postgresql
README.SBo  postgresql-14.22.tar.gz  postgresql.info         rc.postgresql.new     slack-desc
root@Sara:~/postgresql#
```

```
root@Sara:~/postgresql# ./postgresql.SlackBuild
```

```
root@Sara:~/postgresql# installpkg /tmp/postgresql-14.22-i586-1_SBo.tgz
Verifying package postgresql-14.22-i586-1_SBo.tgz.
Installing package postgresql-14.22-i586-1_SBo.tgz:
PACKAGE DESCRIPTION:
# postgresql (object-relational database management system)
#
# PostgreSQL is an advanced object-relational database management
# system (ORDBMS) based on POSTGRES. With more than 15 years of
# development history, it is quickly becoming the de facto
# database for enterprise level open source solutions.
#
# Homepage: https://www.postgresql.org
#
Executing install script for postgresql-14.22-i586-1_SBo.tgz.
Package postgresql-14.22-i586-1_SBo.tgz installed.
root@Sara:~/postgresql# A
```

```
root@Sara:~/postgresql# mkdir -p /var/lib/pgsql/data
root@Sara:~/postgresql# chown postgres:postgres /var/lib/pgsql/data
root@Sara:~/postgresql# su - postgres -c "initdb -D /var/lib/pgsql/data"
The files belonging to this database system will be owned by user "postgres".
This user must also own the server process.

The database cluster will be initialized with locales
  COLLATE:  C
  CTYPE:    en_US.UTF-8
  MESSAGES: en_US.UTF-8
  MONETARY: en_US.UTF-8
  NUMERIC:  en_US.UTF-8
  TIME:     en_US.UTF-8
The default database encoding has accordingly been set to "UTF8".
The default text search configuration will be set to "english".

Data page checksums are disabled.

fixing permissions on existing directory /var/lib/pgsql/data ... ok
creating subdirectories ... ok
selecting dynamic shared memory implementation ... posix
selecting default max_connections ... 100
selecting default shared_buffers ... 128MB
selecting default time zone ... America/Bogota
creating configuration files ... ok
running bootstrap script ... ok
performing post-bootstrap initialization ... ok
syncing data to disk ... ok

initdb: warning: enabling "trust" authentication for local connections
You can change this by editing pg_hba.conf or using the option -A, or
--auth-local and --auth-host, the next time you run initdb.
```

```
Executing install script for postgresql-14.22-i586-1_SBo.tgz.
Package postgresql-14.22-i586-1_SBo.tgz installed.
root@Sara:~/postgresql# mkdir -p /var/lib/pgsql/data
root@Sara:~/postgresql# chown postgres:postgres /var/lib/pgsql/data
root@Sara:~/postgresql# su - postgres -c "initdb -D /var/lib/pgsql/data"
The files belonging to this database system will be owned by user "postgres".
This user must also own the server process.

The database cluster will be initialized with locales
  COLLATE:  C
  CTYPE:    en_US.UTF-8
  MESSAGES: en_US.UTF-8
  MONETARY: en_US.UTF-8
  NUMERIC:  en_US.UTF-8
  TIME:     en_US.UTF-8
The default database encoding has accordingly been set to "UTF8".
The default text search configuration will be set to "english".

Data page checksums are disabled.

fixing permissions on existing directory /var/lib/pgsql/data ... ok
creating subdirectories ... ok
selecting dynamic shared memory implementation ... posix
selecting default max_connections ... 100
selecting default shared_buffers ... 128MB
selecting default time zone ... America/Bogota
creating configuration files ... ok
running bootstrap script ... ok
performing post-bootstrap initialization ... ok
syncing data to disk ... ok

initdb: warning: enabling "trust" authentication for local connections
You can change this by editing pg_hba.conf or using the option -A, or
--auth-local and --auth-host, the next time you run initdb.

Success. You can now start the database server using:

    pg_ctl -D /var/lib/pgsql/data -l logfile start

root@Sara:~/postgresql# su - postgres -c "pg_ctl -D /var/lib/pgsql/data start"
waiting for server to start....2026-03-18 15:20:20.704 -05 [31047] LOG:  starting PostgreSQL 14.22 on
i586-slackware-linux-gnu, compiled by gcc (GCC) 11.2.0, 32-bit
2026-03-18 15:20:20.704 -05 [31047] LOG:  listening on IPv6 address "::1", port 5432
2026-03-18 15:20:20.704 -05 [31047] LOG:  listening on IPv4 address "127.0.0.1", port 5432
2026-03-18 15:20:20.706 -05 [31047] LOG:  listening on Unix socket "/tmp/.s.PGSQL.5432"
2026-03-18 15:20:20.708 -05 [31048] LOG:  database system was shut down at 2026-03-18 15:18:52 -05
2026-03-18 15:20:20.709 -05 [31047] LOG:  database system is ready to accept connections
 done
server started
root@Sara:~/postgresql#
```

```
root@Sara:~/postgresql# su postgres -c "initdb -D /var/lib/pgsql/14/data --locale=en_US.UTF-8 -A md5 -
W"
could not change directory to "/root/postgresql": Permission denied
The files belonging to this database system will be owned by user "postgres".
This user must also own the server process.

The database cluster will be initialized with locale "en_US.UTF-8".
The default database encoding has accordingly been set to "UTF8".
The default text search configuration will be set to "english".

Data page checksums are disabled.

Enter new superuser password:
Enter it again:

fixing permissions on existing directory /var/lib/pgsql/14/data ... ok
creating subdirectories ... ok
selecting dynamic shared memory implementation ... posix
selecting default max_connections ... 100
selecting default shared_buffers ... 128MB
selecting default time zone ... America/Bogota
creating configuration files ... ok
running bootstrap script ... ok
performing post-bootstrap initialization ... ok
syncing data to disk ... ok

Success. You can now start the database server using:

    pg_ctl -D /var/lib/pgsql/14/data -l logfile start
```

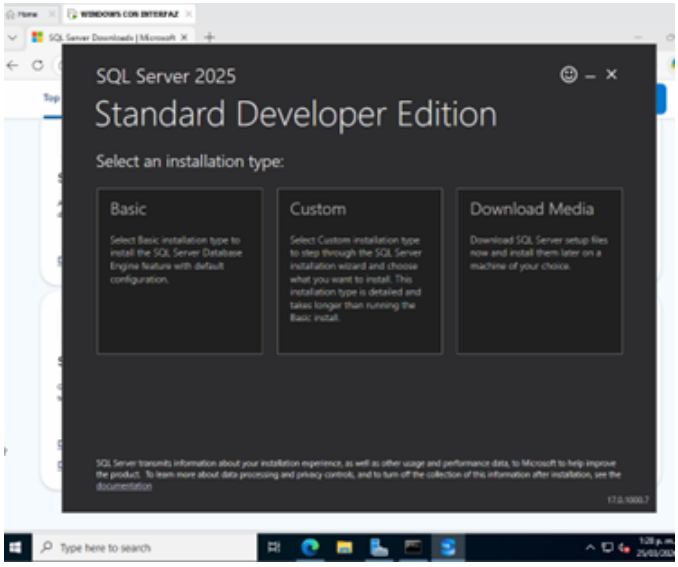
```
root@Sara:~/postgresql# /etc/rc.d/rc.postgresql start
Starting PostgreSQL
PostgreSQL daemon already running
Warning: Missing pid file /var/lib/pgsql/14/data/postmaster.pid
root@Sara:~/postgresql# /etc/rc.d/rc.postgresql stop
Shutting down PostgreSQL...
pg_ctl: PID file "/var/lib/pgsql/14/data/postmaster.pid" does not exist
Is server running?
root@Sara:~/postgresql# /etc/rc.d/rc.postgresql start
Starting PostgreSQL
PostgreSQL daemon already running
Warning: Missing pid file /var/lib/pgsql/14/data/postmaster.pid
root@Sara:~/postgresql# killall -9 postgres
root@Sara:~/postgresql# rm /tmp/.s.PGSQL.5432
root@Sara:~/postgresql# rm /tmp/.s.PGSQL.5432.lock
root@Sara:~/postgresql# /etc/rc.d/rc.postgresql start
Starting PostgreSQL
waiting for server to start.... done
server started
root@Sara:~/postgresql# psql -U postgres
Password for user postgres:
psql (14.22)
Type "help" for help.

postgres=# |
```

```
postgres=# CREATE DATABASE turismo_julian OWNER julian;
CREATE DATABASE
postgres=# REVOKE ALL ON DATABASE turismo_julian FROM PUBLIC;
REVOKE
postgres=# GRANT ALL PRIVILEGES ON DATABASE turismo_julian TO julian;
GRANT
postgres=# CREATE TABLE departamentos (id SERIAL PRIMARY KEY, nombre VARCHAR(50) NOT NULL);
CREATE TABLE
postgres=# CREATE TABLE sitios (id SERIAL PRIMARY KEY, nombre VARCHAR(100) NOT NULL, id_depto INT REFERENCES departamentos(id));
CREATE TABLE
postgres=# CREATE TABLE actividades (id SERIAL PRIMARY KEY, descripcion TEXT, id_sitio INT REFERENCES sitios(id));
CREATE TABLE
postgres=# INSERT INTO departamentos (nombre) VALUES ('Antioquia'), ('Amazonas');
INSERT 0 2
postgres=# INSERT INTO sitios (nombre, id_depto) VALUES ('Piedra del Peñol', 1), ('Puerto Nariño', 2);
INSERT 0 2
postgres=# INSERT INTO actividades (descripcion, id_sitio) VALUES ('Subir los 740 escalones', 1), ('Observación de delfines rosados', 2);
INSERT 0 2
```

Instalación de la base de datos en Windows

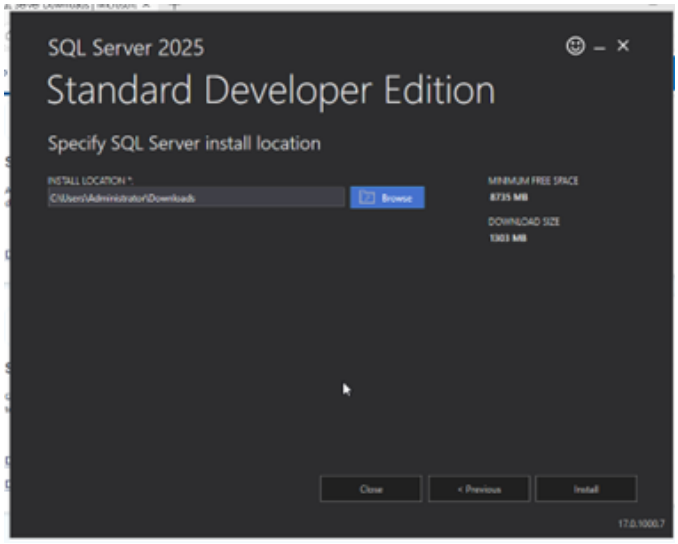
Creación de usuarios y su contenido (Slackware)

```
postgres=# CREATE USER sara WITH PASSWORD 'sara123';
CREATE ROLE
postgres=# CREATE USER julian WITH PASSWORD 'julian123';
CREATE ROLE
postgres=# |
```

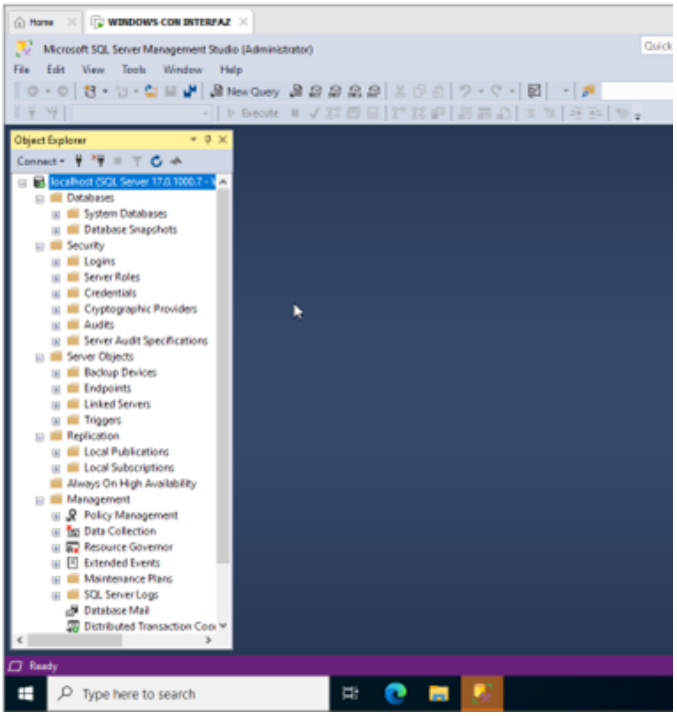



```
postgres=# CREATE DATABASE turismo_sara OWNER sara;
CREATE DATABASE
postgres=# REVOKE ALL ON DATABASE turismo_sara FROM PUBLIC;
REVOKE
postgres=# GRANT ALL PRIVILEGES ON DATABASE turismo_sara TO sara;
GRANT
postgres=# |
```

```
postgres=# \c turismo_sara
You are now connected to database "turismo_sara" as user "postgres".
turismo_sara=# CREATE TABLE departamentos (
    id SERIAL PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL
);
CREATE TABLE
turismo_sara=# CREATE TABLE sitios (
    id SERIAL PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    id_depto INT REFERENCES departamentos(id)
);
CREATE TABLE
turismo_sara=# CREATE TABLE actividades (
    id SERIAL PRIMARY KEY,
    descripcion TEXT,
    id_sitio INT REFERENCES sitios(id)
);
CREATE TABLE
turismo_sara=# |
```




```
turismo_sara=# INSERT INTO departamentos (nombre) VALUES ('Bolivar'), ('Quindio');
INSERT 0 2
turismo_sara=# INSERT INTO sitios (nombre, id_depto) VALUES ('Ciudad Amurallada', 1), ('Valle del Coco
ra', 2);
INSERT 0 2
turismo_sara=# INSERT INTO actividades (descripcion, id_sitio) VALUES ('Tour histórico por las muralla
s', 1), ('Caminata por los bosques de palma', 2);
INSERT 0 2
turismo_sara=# |
```




```
$ initdb -D /var/db/pgsql/data
The files belonging to this database system will be owned by user "postgres".
This user must also own the server process.

The database cluster will be initialized with locale "C".
The default database encoding has accordingly been set to "SQL_ASCII".
The default text search configuration will be set to "english".

Data page checksums are disabled.

fixing permissions on existing directory /var/db/pgsql/data ... ok
creating subdirectories ... ok
selecting dynamic shared memory implementation ... posix
selecting default max_connections ... 20
selecting default shared_buffers ... 128MB
selecting default time zone ... UTC
creating configuration files ... ok
running bootstrap script ... ok
performing post-bootstrap initialization ... ok
syncing data to disk ... ok

initdb: warning: enabling "trust" authentication for local connections
You can change this by editing pg_hba.conf or using the option -A, or
--auth-local and --auth-host, the next time you run initdb.

Success. You can now start the database server using:

    pg_ctl -D /var/db/pgsql/data -l logfile start

$ |
```

```
$ exit
# chown -R postgres:postgres /var/db/pgsql
# su - postgres
$ pg_ctl -D /var/db/pgsql/data -l /var/db/pgsql/logfile start
waiting for server to start.... done
server started
$
```

Creación de usuarios y su contenido (NetBSD)

```
postgres=# CREATE USER Sara WITH PASSWORD 'sara123';
CREATE ROLE
postgres=# CREATE USER Julian WITH PASSWORD 'julian123';
CREATE ROLE
postgres=# CREATE USER Diego WITH PASSWORD 'diego123';
CREATE ROLE
postgres=# CREATE DATABASE series_sara OWNER Sara;
CREATE DATABASE
postgres=# CREATE DATABASE series_julian OWNER Julian;
CREATE DATABASE
postgres=# CREATE DATABASE series_diego OWNER Diego;
CREATE DATABASE
postgres=# REVOKE ALL ON DATABASE series_sara FROM PUBLIC;
REVOKE
postgres=# REVOKE ALL ON DATABASE series_julian FROM PUBLIC;
REVOKE
postgres=# REVOKE ALL ON DATABASE series_diego FROM PUBLIC;
REVOKE
postgres=#
```

Instalación de la base de datos en NetBSD

```
# pkgin update
processing remote summary (https://cdn.netbsd.org/pub/pkgsrc/packages/NetBSD/x86_64/10.0/All)...
pkg_summary.bz2                                          100% 4194KB 174.8KB/s   00:24
# pkgin install postgresql14-server
calculating dependencies...done.

5 packages to install:
  libxml2-2.14.6 lz4-1.10.0 postgresql14-client-14.20 postgresql14-server-14.20 xmlcatmgr-2.2nb1

0 to remove, 0 to refresh, 0 to upgrade, 5 to install
86748K to download, 477M of additional disk space will be used

proceed ? [Y/n] y
[1/5] libxml2-2.14.6.tgz                                 100%  892KB 892.3KB/s   00:00
[2/5] lz4-1.10.0.tgz                                     100%  135KB 134.8KB/s   00:00
[3/5] postgresql14-client-14.20.tgz                      100% 2884KB   2.8MB/s   00:01
[4/5] postgresql14-server-14.20.tgz                      100% 4738KB   1.5MB/s   00:03
[5/5] xmlcatmgr-2.2nb1.tgz                               100%   26KB  25.8KB/s   00:00
[1/5] installing lz4-1.10.0...
[2/5] installing libxml2-2.14.6...
xmlcatmgr-2.2nb1: copying /usr/pkg/share/examples/xmlcatmgr/catalog.etc.sgml to /usr/pkg/etc/sgml/cata
log
xmlcatmgr-2.2nb1: copying /usr/pkg/share/examples/xmlcatmgr/catalog.etc.xml to /usr/pkg/etc/xml/catalo
g
```

```
# groupadd postgres
# useradd -g postgres -d /var/db/pgsql -s /bin/sh postgres
# chown postgres:postgres /var/db/pgsql/data
# su - postgres
$
```

```
postgres=# \c series_sara
You are now connected to database "series_sara" as user "postgres".
series_sara=# CREATE TABLE generos (
series_sara(#     id_genero   SERIAL PRIMARY KEY,
series_sara(#     nombre      VARCHAR(50) NOT NULL,
series_sara(#     descripcion VARCHAR(100)
series_sara(# );

CREATE TABLE series (
    id_serie     SERIAL PRIMARY KEY,
    titulo       VARCHAR(150) NOT NULL,
    plataforma   VARCHAR(50),
    anio_inicio  INT,
    id_genero    INT REFERENCES generos(id_genero)
);

CREATE TABLE seguimiento (
    id_seguimiento  SERIAL PRIMARY KEY,
    id_serie        INT REFERENCES series(id_serie),
    temporada_actual INT,
    episodio_actual  INT,
    estado           VARCHAR(20) CHECK (estado IN ('Viendo', 'Pendiente', 'Terminada'))
);CREATE TABLE
series_sara=#
series_sara=# CREATE TABLE series (
series_sara(#     id_serie     SERIAL PRIMARY KEY,
series_sara(#     titulo       VARCHAR(150) NOT NULL,
series_sara(#     plataforma   VARCHAR(50),
series_sara(#     anio_inicio  INT,
series_sara(#     id_genero    INT REFERENCES generos(id_genero)
series_sara(# );
CREATE TABLE
series_sara=#
series_sara=# CREATE TABLE seguimiento (
series_sara(#     id_seguimiento  SERIAL PRIMARY KEY,
series_sara(#     id_serie        INT REFERENCES series(id_serie),
series_sara(#     temporada_actual INT,
series_sara(#     episodio_actual  INT,
series_sara(#     estado           VARCHAR(20) CHECK (estado IN ('Viendo', 'Pendiente', 'Terminada'))
series_sara(# );
CREATE TABLE
series_sara=# INSERT INTO generos (nombre, descripcion) VALUES
series_sara-# ('Drama',    'Series dramticas'),
series_sara-# ('Comedia',  'Series de humor'),
series_sara-# ('Thriller', 'Series de suspenso');

INSERT INTO series (titulo, plataforma, anio_inicio, id_genero) VALUES
series_sara=#
series_sara=# INSERT INTO series (titulo, plataforma, anio_inicio, id_genero) VALUES
```

```
INSERT INTO series (titulo, plataforma, anio_inicio, iINSERT 0 3
series_sara=#
series_sara=# INSERT INTO series (titulo, plataforma, anio_inicio, id_genero) VALUES
series_sara-# ('Breaking Bad',    'Netflix',      2008, 3),
series_sara-# ('Friends',         'HBO Max',      1994, 2),
series_sara-# ('The Crown',       'Netflix',      2016, 1),
series_sara-# ('Stranger Things', 'Netflix',      2016, 3),
series_sara-# ('The Office',      'Peacock',      2005, 2);
INSERT 0 5
series_sara=#
series_sara=# INSERT INTO seguimiento (id_serie, temporada_actual, episodio_actual, estado) VALUES
series_sara-# (1, 5, 16, 'Terminada'),
series_sara-# (2, 3, 10, 'Viendo'),
series_sara-# (3, 4, 5,  'Pendiente'),
series_sara-# (4, 4, 9,  'Viendo'),
series_sara-# (5, 2, 6,  'Terminada');
INSERT 0 5
```

```
postgres=# GRANT ALL PRIVILEGES ON DATABASE series_sara TO Sara;
GRANT
postgres=# GRANT ALL PRIVILEGES ON DATABASE series_julian TO Julian;
GRANT
postgres=# GRANT ALL PRIVILEGES ON DATABASE series_diego TO Diego;
GRANT
postgres=# |
```

Inicio automático de la base de datos en Slackware

```
postgres=# \c series_julian
You are now connected to database "series_julian" as user "postgres".
series_julian=# CREATE TABLE generos (
series_julian(#     id_genero   SERIAL PRIMARY KEY,
series_julian(#     nombre      VARCHAR(50) NOT NULL,
series_julian(#     descripcion VARCHAR(100)
series_julian(# );
CREATE TABLE
series_julian=# CREATE TABLE series (
series_julian(#     id_serie    SERIAL PRIMARY KEY,
series_julian(#     titulo      VARCHAR(150) NOT NULL,
series_julian(#     plataforma  VARCHAR(50),
series_julian(#     anio_inicio INT,
series_julian(#     id_genero   INT REFERENCES generos(id_genero)
series_julian(# );
CREATE TABLE
series_julian=# CREATE TABLE series (
series_julian(#     id_serie    SERIAL PRIMARY KEY,
series_julian(#     titulo      VARCHAR(150) NOT NULL,
series_julian(#     plataforma  VARCHAR(50),
series_julian(#     anio_inicio INT,
series_julian(#     id_genero   INT REFERENCES generos(id_genero)
);    id_genero   INT REFERENCES generos(id_genero)
series_julian(# );
ERROR:  relation "series" already exists
series_julian=# CREATE TABLE seguimiento (
series_julian(#     id_seguimiento  SERIAL PRIMARY KEY,
series_julian(#     id_serie        INT REFERENCES series(id_serie),
series_julian(#     temporada_actual INT,
series_julian(#     episodio_actual  INT,
series_julian(#     estado          VARCHAR(20) CHECK (estado IN ('Viendo', 'Pendiente', 'Terminada'))
series_julian(# );
CREATE TABLE
series_julian=# INSERT INTO generos (nombre, descripcion) VALUES
series_julian-# ('Accion', 'Series de accion'),
series_julian-# ('Sci-Fi', 'Ciencia ficcion'),
series_julian-# ('Terror', 'Series de miedo');

INSERT INTO series (titulo, plataforma, anio_inicio, id_genero) VALUES
('The Witcher', 'Netflix', 2019, 1),
('Black Mirror', 'Netflix', 2011, 2),
('The Walking INSERT 0 3
series_julian=#
series_julian=# INSERT INTO series (titulo, plataforma, anio_inicio, id_genero) VALUES
series_julian-# ('The Witcher', 'Netflix', 2019, 1),
series_julian-# ('Black Mirror', 'Netflix', 2011, 2),
series_julian-# ('The Walking Dead', 'AMC', 2010, 3),
series_julian-# ('Westworld', 'HBO Max', 2016, 2),
series_julian-# ('Squid Game', 'Netflix', 2021, 1);
INSERT 0 5
```

```
root@Sara:~# ls -l /etc/rc.d/rc.postgresql
-rwxr-xr-x 1 root root 4981 Mar 18 15:12 /etc/rc.d/rc.postgresql*
root@Sara:~# /etc/rc.d/rc.postgresql start
Starting PostgreSQL
pg_ctl: another server might be running; trying to start server anyway
waiting for server to start.... done
server started
root@Sara:~# ps aux | grep postgres
postgres  1110  0.0  0.1  30380  4860 ?        Sl   12:15   0:00 /usr/bin/gnome-keyring-daemon --daemo
nize --login
postgres  1121  1.0  0.4 161252 16712 ?        Ss   12:15   0:00 /usr/lib/postgresql/14/bin/postgres -
D /var/lib/pgsql/14/data -p 5432
postgres  1123  0.0  0.0 161252  3196 ?        Ss   12:15   0:00 postgres: checkpointer
postgres  1124  0.0  0.1 161252  5208 ?        Ss   12:15   0:00 postgres: background writer
postgres  1125  0.0  0.2 161252  8736 ?        Ss   12:15   0:00 postgres: walwriter
postgres  1126  0.0  0.1 161664  6428 ?        Ss   12:15   0:00 postgres: autovacuum launcher
postgres  1127  0.0  0.0  18312  3836 ?        Ss   12:15   0:00 postgres: stats collector
postgres  1128  0.0  0.1 161556  6576 ?        Ss   12:15   0:00 postgres: logical replication launche
r
root      1130  0.0  0.0   3864  1892 pts/0    S+   12:15   0:00 grep postgres
root@Sara:~# |
```

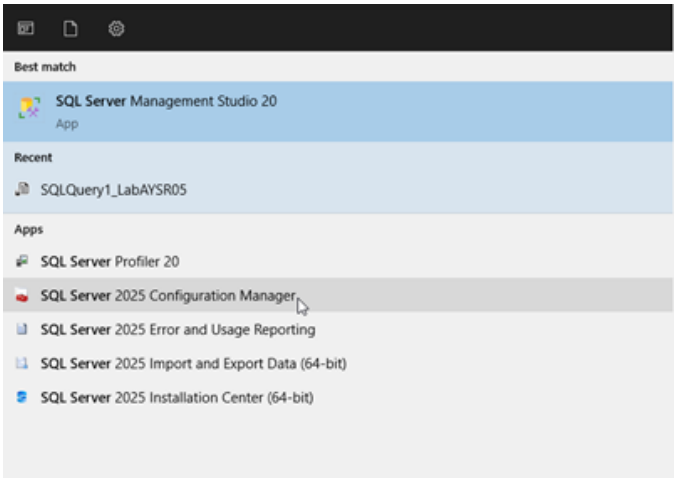
```
root@Sara:~# nano /etc/rc.d/rc.local
```

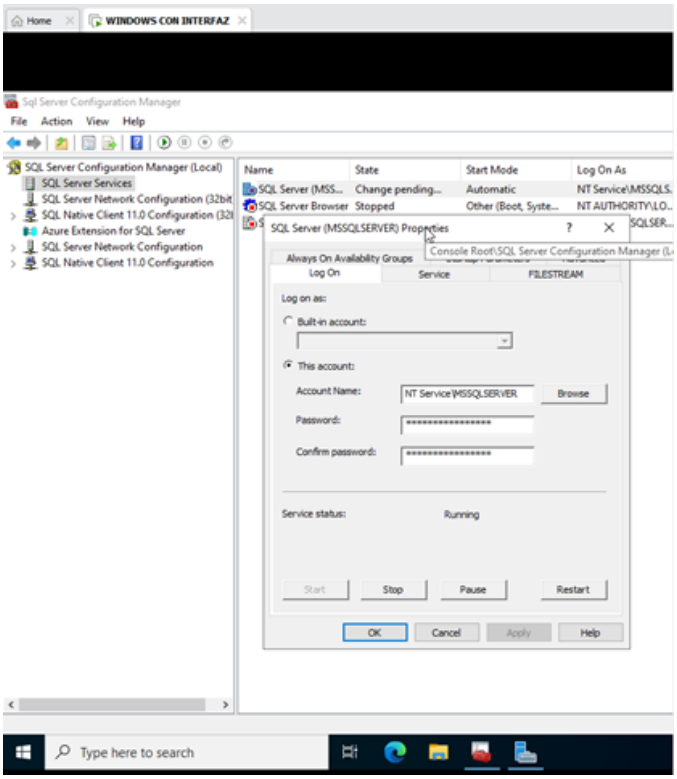
```
series_julian=# \c series_diego
You are now connected to database "series_diego" as user "postgres".
series_diego=# CREATE TABLE generos (
series_diego(#     id_genero SERIAL PRIMARY KEY,
series_diego(#     nombre VARCHAR(50) NOT NULL,
series_diego(#     descripcion VARCHAR(100)
series_diego(# );

CREATE TABLE series (
    id_serie SERIAL PRIMARY KEY,
    titulo VARCHAR(150) NOT NULL,
    plataforma VARCHAR(50),
    anio_inicio INT,
    id_genero INT REFERENCES generos(id_genero)
);

CREATE TABLE seguimiento (
    id_seguimiento SERIAL PRIMARY KEY,
    id_serie INT REFERENCES series(id_serie),
    temporada_actual INT,
    episodio_actual INT,
    estado VARCHAR(20) CHECK (estado IN ('Viendo','Pendiente','Terminada'))
);CREATE TABLE
series_diego=#
series_diego=# CREATE TABLE series (
series_diego(#     id_serie SERIAL PRIMARY KEY,
series_diego(#     titulo VARCHAR(150) NOT NULL,
series_diego(#     plataforma VARCHAR(50),
series_diego(#     anio_inicio INT,
series_diego(#     id_genero INT REFERENCES generos(id_genero)
series_diego(# );
CREATE TABLE
series_diego=#
series_diego=# CREATE TABLE seguimiento (
series_diego(#     id_seguimiento SERIAL PRIMARY KEY,
series_diego(#     id_serie INT REFERENCES series(id_serie),
series_diego(#     temporada_actual INT,
series_diego(#     episodio_actual INT,
series_diego(#     estado VARCHAR(20) CHECK (estado IN ('Viendo','Pendiente','Terminada'))
series_diego(# );
CREATE TABLE
series_diego=# INSERT INTO generos (nombre, descripcion) VALUES
series_diego-# ('Fantasia', 'Mundos fantasticos'),
series_diego-# ('Crimen',   'Series policiales'),
series_diego-# ('Animacion','Series animadas');

INSERT INTO series (titulo, plataforma, anio_inicio, iINSERT 0 3
series_diego=#
series_diego=# INSERT INTO series (titulo, plataforma, anio_inicio, id_genero) VALUES
```

```
Command Prompt - ssh sara@
  GNU nano 6.0                    /etc/rc.d/rc.local
#!/bin/bash
#
# /etc/rc.d/rc.local:  Local system initialization script.
#
# Put any local startup commands in here.  Also, if you have
# anything that needs to be run at shutdown time you can
# make an /etc/rc.d/rc.local_shutdown script and put those
# commands in there.

ip route add 10.2.65.1 dev eth0
ip route add default via 10.2.65.1 dev eth0

/etc/rc.d/rc.postgresql start
```

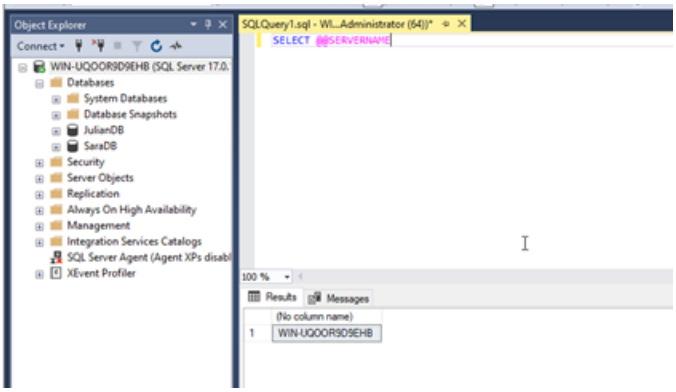
```
root@Sara:~# chmod +x /etc/rc.d/rc.local
root@Sara:~# |
```

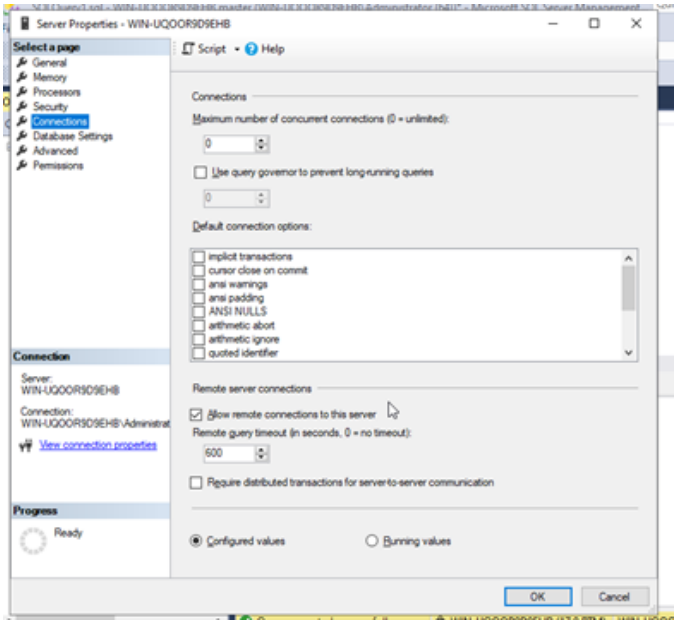
Inicio automático de la base de datos en Windows

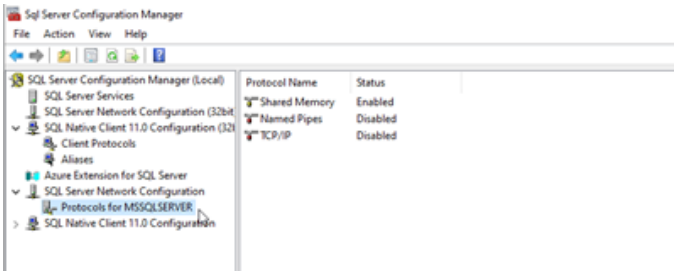


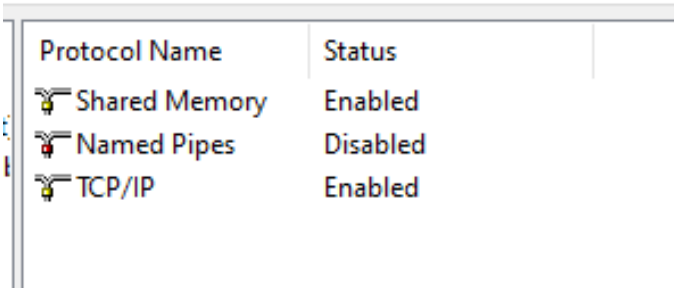


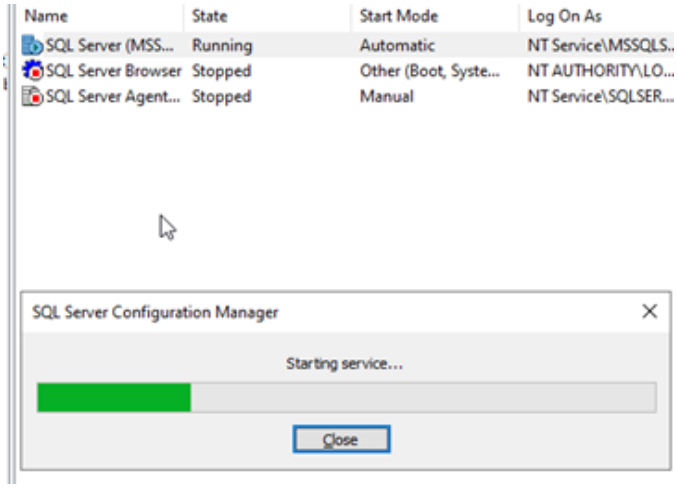
Conexión con DBeaver en Windows

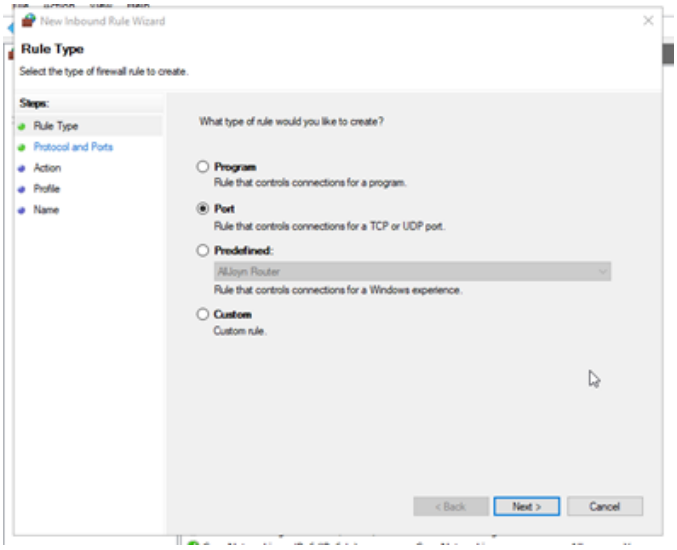


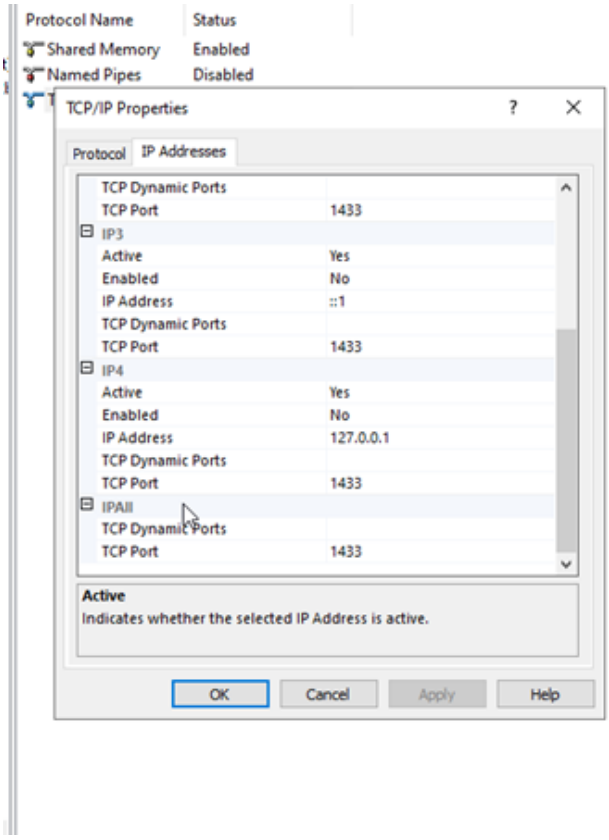


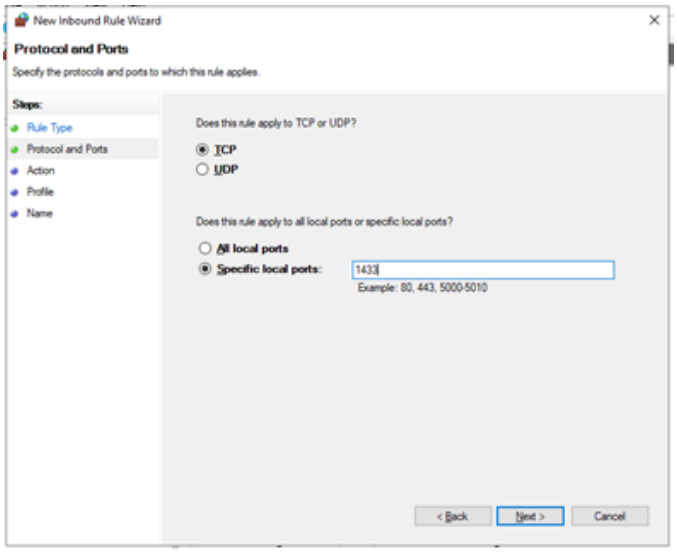


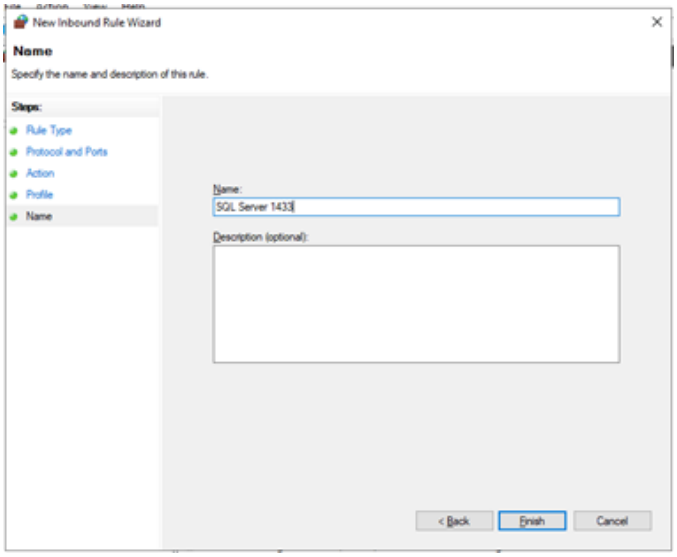

Protocol Name
Status
Shared Memory
Enabled
Named Pipes
Disabled
TCP/IP
Enabled

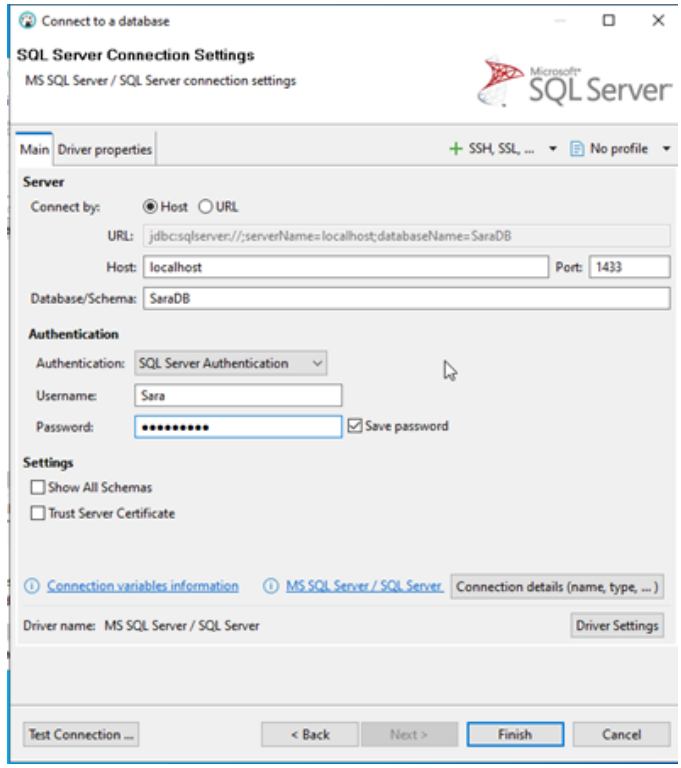

Name
State
Start Mode
Log On As
SQL Server (MSS...
Running
Automatic
NT Service\MSSQLS...
SQL Server Browser
Stopped
Other (Boot, Syste...
NT AUTHORITY\LO...
SQL Server Agent...
Stopped
Manual
NT Service\SQLSER...
SQL Server Configuration Manager
Starting service...
Close


New Inbound Rule Wizard
Rule Type
Select the type of firewall rule to create.
Steps:
Rule Type
Protocol and Ports
Action
Profile
Name
What type of rule would you like to create?
Program
Rule that controls connections for a program.
Port
Rule that controls connections for a TCP or UDP port.
Predefined:
AllJoyn Router
Rule that controls connections for a Windows experience.
Custom
Custom rule.
< Back
Next >
Cancel

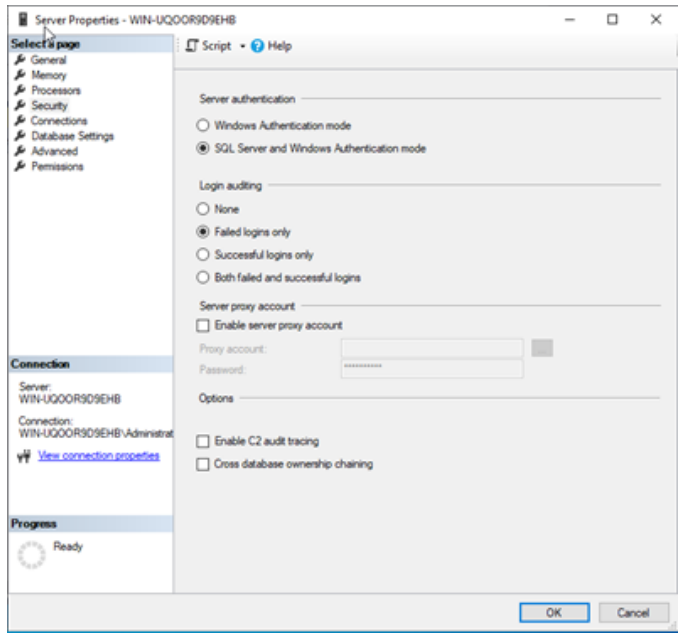

Protocol Name
Status
Shared Memory
Enabled
Named Pipes
Disabled
TCP/IP Properties
Protocol
IP Addresses
TCP Dynamic Ports
TCP Port
1433
IP3
Active
Yes
Enabled
No
IP Address
::1
TCP Dynamic Ports
TCP Port
1433
IP4
Active
Yes
Enabled
No
IP Address
127.0.0.1
TCP Dynamic Ports
TCP Port
1433
IPAll
TCP Dynamic Ports
TCP Port
1433
Active
Indicates whether the selected IP Address is active.
OK
Cancel
Apply
Help

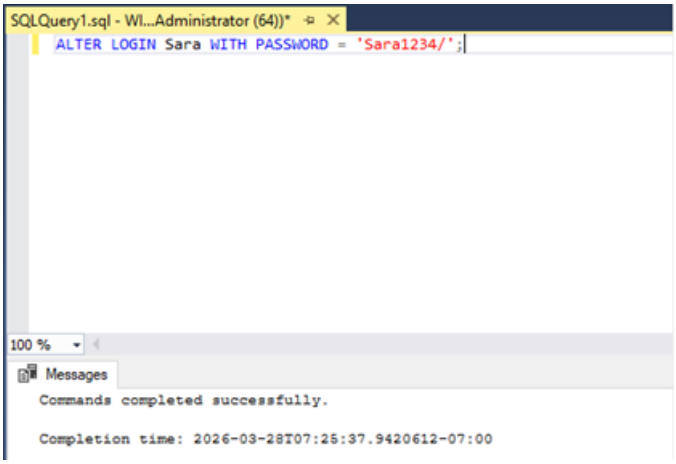

New Inbound Rule Wizard
Protocol and Ports
Specify the protocols and ports to which this rule applies.
Steps:
Rule Type
Protocol and Ports
Action
Profile
Name
Does this rule apply to TCP or UDP?
TCP
UDP
Does this rule apply to all local ports or specific local ports?
All local ports
Specific local ports:
1433
Example: 80, 443, 5000-5010
< Back
Next >
Cancel

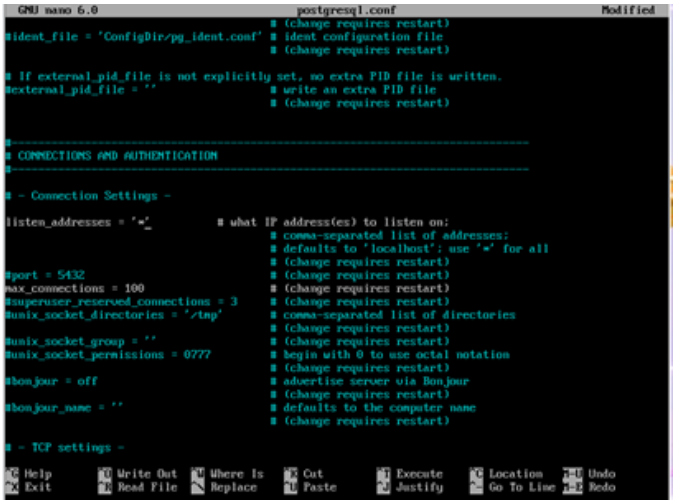

New Inbound Rule Wizard
Name
Specify the name and description of this rule.
Steps:
Rule Type
Protocol and Ports
Action
Profile
Name
Name:
SQL Server 1433
Description (optional):
< Back
Finish
Cancel


Connect to a database
SQL Server Connection Settings
MS SQL Server / SQL Server connection settings
Microsoft SQL Server
Main
Driver properties
SSH, SSL, ...
No profile
Server
Connect by:
Host
URL
URL:
jdbc:sqlserver://;serverName=localhost;databaseName=SaraDB
Host:
localhost
Port:
1433
Database/Schema:
SaraDB
Authentication
Authentication:
SQL Server Authentication
Username:
Sara
Password:
Save password
Settings
Show All Schemas
Trust Server Certificate
Connection variables information
MS SQL Server / SQL Server
Connection details (name, type, ...)
Driver name: MS SQL Server / SQL Server
Driver Settings
Test Connection ...
< Back
Next >
Finish
Cancel

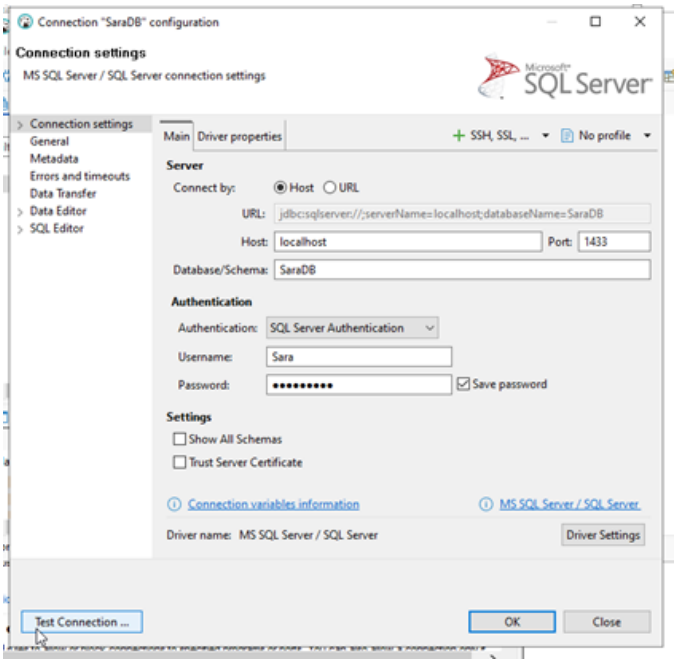
Connect to a database
Select your database
Create new database connection. Find your database driver in the list below.
Type part of database/driver name to filter
Sort by:
Title
Score
All
Popular
SQL
NoSQL
Analytical
Files
Embedded
Timeseries
Hadoop / BigData
Full-text search
Graph databases
SQLite
Db2 for LUW
MariaDB
MySQL
Oracle
PostgreSQL
SQL Server
Altibase
Apache Calcite Avatica
Apache Drill
Apache Hive 2
Apache Hive 4+
Apache Ignite
Apache IoTDB
Apache Kylin
Apache Kyuubi
Hadoop / Apache Drill
No saved connections yet
Apache Drill JDBC
Test Connection ...
< Back
Next >
Finish
Cancel


Server Properties - WIN-UQOOR9D9EHB
Select a page
General
Memory
Processors
Security
Connections
Database Settings
Advanced
Permissions
Script
Help
Server authentication
Windows Authentication mode
SQL Server and Windows Authentication mode
Login auditing
None
Failed logins only
Successful logins only
Both failed and successful logins
Server proxy account
Enable server proxy account
Proxy account:
Password:
Options
Enable C2 audit tracing
Cross database ownership chaining
Connection
Server:
WIN-UQOOR9D9EHB
Connection:
View connection properties
Progress
Ready
OK
Cancel

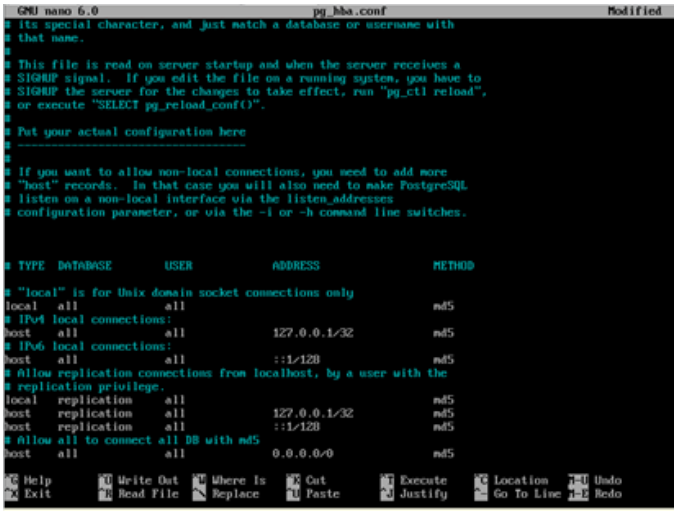





```
root@Sara:/var/lib/pgsql/14/data# cp pg_hba.conf pg_hba.conf.bak
```




```
root@Sara:/var/lib/pgsql/14/data# nano pg_hba.conf_
```



Proceso de inicio automático en Windows (adicional)

```
root@Sara:/var/lib/pgsql/14/data# cp postgresql.conf postgresql.conf.bak_
```

```
root@Sara:/var/lib/pgsql/14/data# /etc/rc.d/rc.postgresql restart
Restarting PostgreSQL...
waiting for server to shut down.... done
server stopped
waiting for server to start.... done
server started
root@Sara:/var/lib/pgsql/14/data# _
```

```
root@Sara:/var/lib/pgsql/14/data# nano postgresql.conf_
```

```
root@Sara:/var/lib/pgsql/14/data# iptables -I INPUT -p tcp -s 192.168.56.1 --dport 5432 -j ACCEPT
root@Sara:/var/lib/pgsql/14/data# _
```

```
root@Sara:/var/lib/pgsql/14/data# iptables-save > /etc/iptables.rules
root@Sara:/var/lib/pgsql/14/data# ss -ltnp | grep 5432
root@Sara:/var/lib/pgsql/14/data# ss -ltnp | grep 5432
LISTEN 0      244                          0.0.0.0:5432      0.0.0.0:*    users:(("postgres",pid=1
155,fd=5))
LISTEN 0      244                             [::]:5432         [::]:*    users:(("postgres",pid=1
155,fd=6))
root@Sara:/var/lib/pgsql/14/data# _
```